

Chapter 8: Introduction to Stan

Shravan Vasishth

2026-03-26

Contents

1	Textbook	2
2	Introduction	2
3	Stan syntax	7
3.1	A first simple example with Stan: Normal likelihood	8
4	Aside: What does <code>target</code> do?	9
4.1	Incrementing the log probability function (<code>target</code>) vs. using the <code>~</code> notation.	11
5	Another simple example: Cloze probability with Stan with the binomial likelihood	16
6	Regression models in Stan	18
7	A first linear regression in Stan: Does attentional load affect pupil size?	18
7.1	Matrix, vector, or array in Stan?	24
7.2	Interactions in Stan: Does attentional load interact with trial number affecting pupil size?	25
8	Logistic regression in Stan: Does set size and trial affect free recall?	29

1 Textbook

Introduction to Bayesian Data Analysis for Cognitive Science

Nicenboim, Schad, Vasishth

- Online version: <https://bruno.nicenboim.me/bayescogsci/>
- Source code: <https://github.com/bnicenboim/bayescogsci>
- Physical book: [here](#)

Be sure to read the textbook's chapter 8 in addition to watching this lecture.

2 Introduction

There are several software packages available for Bayesian modeling; for example:

- R-INLA
- WinBUGS
- JAGS
- PyMC
- Turing
- Stan

Stan takes advantage of the fact that the shape of the posterior distribution is completely determined by priors and the likelihood we have defined; by that we mean that we know the analytical form of the unnormalized posterior, which is the upper part of the Bayes rule (abbreviated below as q). This is because the denominator, or marginal likelihood, “only” constitutes a normalizing constant:

$$p(\Theta|y) = \frac{p(y|\Theta) \cdot p(\Theta)}{p(y)} \quad (1)$$

$$q(\Theta|y) = p(y|\Theta) \cdot p(\Theta) \quad (2)$$

Thus the unnormalized posterior is proportional ($\propto$) to the posterior distribution:

$$q(\Theta|y) \propto p(\Theta|y) \quad (3)$$

The Stan sampler uses Hamiltonian dynamics and treats the vector of parameters, Θ (that could range from a vector containing a couple of parameters, e.g., $[\mu, \sigma]$, to a vector of hundreds of parameters in hierarchical models), as the position of a frictionless particle that glides on the *negative logarithm of the unnormalized posterior*. That means that high probability places are valleys and low probability places are peaks in this space.¹ However, Stan doesn't just let the particle glide until the bottom of this space. If we let that happen, we would find the mode of the posterior distribution, rather than samples. Stan uses a complex algorithm to determine the weight of the particle and the momentum that we apply to it, as well as when to stop the particle trajectory to take a sample. Because we need to know the velocity of this particle, Stan needs to be able to calculate the derivative of the log unnormalized posterior with respect to the parameters (recall that velocity is the first derivative of position). This means that if the parameter space is differentiable and relatively smooth (i.e., does not have any big break or sharp angle), and if the parameters of the Stan algorithm are well adjusted—as should happen in the warm-up period—these samples are going to represent samples of the true posterior distribution. Bear in mind that the geometry of the posterior has a big influence on whether the algorithm will converge (fast) or not: If the space is very flat, because there isn't much data and the priors are not informative, then the particle may need to glide for a long time before it gets to a high probability area, that is a valley; if there are several valleys (multimodality) the particle may never leave the vicinity of one of them; and if the space is funnel shaped, the particle may never explore the funnel. One of the reasons for the difficulties in exploring complicated spaces is that the continuous path of the “particle” is discretized and divided into steps, and the *step size* is optimized for the entire posterior space. In

¹ In the specific case of a model with two parameters, e.g., $[\mu, \sigma]$, the physical analogy works quite well: The $[x, y]$ coordinates of the particle would be determined by $[\mu, \sigma]$, and its z coordinate would be established by the height of the unnormalized posterior.

spaces that are too complex, such as a funnel, a step size might be too small to explore the wide part of the funnel, but too large to explore the narrow part; we will deal with this problem in section [@ref\(sec-uncorrstan\)](#).

Although our following example assumes a vector of two parameters and thus a simple geometry, real world examples can easily have hundreds of parameters defining an unnormalized posterior space with hundreds of dimensions.

One question that might arise here is the following: Given that we already know the shape of the posterior, why do we need samples? After all, the posterior is just the unnormalized posterior multiplied by some number, the normalizing constant.

To make this discussion concrete, let's say that we have a subject that participates in a memory test, and in each trial we get a noisy score from their true working memory score. We assume that at each trial, the score is elicited with normally distributed noise. If we want to estimate the score and how much the noise makes it vary from trial to trial, we are assuming a normal likelihood and we want to estimate its mean and standard deviation.

We will use simulated data produced by a normal distribution with a true mean of 3 and a true standard deviation of 10:

```
Y <- rnorm(n = 100, mean = 3, sd = 10)
head(Y)

## [1] 16.643028  1.839367  7.534832 14.231758  3.871526 -3.137406
```

As always, given our prior knowledge, we decide on priors. In this case, we use a log-normal prior for the standard deviation, σ , since it can only be positive, but except for that, the prior distributions are quite arbitrary in this example.

$$\begin{aligned}\mu &\sim \text{Normal}(0, 20) \\ \sigma &\sim \text{LogNormal}(3, 1)\end{aligned}\tag{4}$$

The unnormalized posterior will be the product of the

likelihood of each data point times the prior for each parameter:

$$q(\mu, \sigma | y) = \prod_n^{100} \text{Normal}(y_n | \mu, \sigma) \cdot \text{Normal}(\mu | 0, 20) \cdot \text{LogNormal}(\sigma | 3, 1) \quad (5)$$

where $y = 16.6430283, 1.8393673, \dots$

We can also define the unnormalized posterior, $q(\cdot)$, as a function in R:

```
q <- function(mu, sigma, y) {
  dnorm(x = mu, mean = 0, sd = 20) *
  dlnorm(x = sigma, mean = 3, sd = 1) *
  prod(dnorm(x = y, mean = mu, sd = sigma))
}
```

For example, if we want to know the unnormalized posterior density for the vector of parameters $[\mu, \sigma] = [0, 5]$, we do the following:

```
q(mu = 0, sigma = 5, y = Y)
## [1] 2.639441e-211
```

The shape of the unnormalized posterior density is completely defined and it will look like Figure 1.

Why is the shape of the unnormalized posterior density not enough? The main reason is that unless we already know which probability distribution we are dealing with (e.g., normal, Bernoulli, etc.) or we can easily integrate it (which can only be done in simpler cases) we cannot do much with the analytical form of the unnormalized posterior: We cannot calculate credible intervals, or know how likely it is that the true score is above or below zero, and even the mean of the posterior is impossible to calculate. This is because the unnormalized posterior distribution represents the shape of the posterior distribution. With just the shape of an unknown distribution, we can only answer the following question: What is the most (or least) likely value of the vector of parameters? We can answer this question by searching for the highest (or lowest)

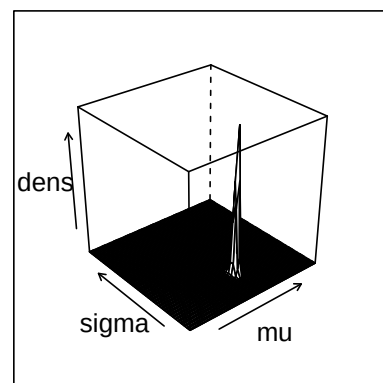


Figure 1: The unnormalized posterior.

place in that shape. This leads us to the maximum a posteriori (MAP) estimate (i.e., the highest point in Figure 1, or the lowest point of the negative log unnormalized log posterior), which is in a sense the “Bayesian counterpart” of the penalized maximum likelihood estimate (MLE).

However, it is not truly Bayesian since it doesn’t take into account the uncertainty of the posterior. Even calculating the mode is not always trivial. In simple cases as this one, one can calculate it analytically; but in more complex cases relatively complicated algorithms are needed. (If we can recognize the shape as a distribution, we are in a different situation. In that case, we might know already the formulas for the expectation, variance, etc. This is what we did in chapter 3, but this is an unusual situation in realistic analyses.) As we mentioned before, if we want to get posterior density values, we need the denominator of the Bayes rule (or marginal likelihood), $p(y)$, which requires integrating the unnormalized posterior. Even this is not too useful if we want to communicate findings, almost every summary statistic requires us to solve more integrals, and except for a handful of cases, these integrals might not have an analytical solution.

If we want to be able to calculate summary statistics of the posterior distribution (mean, quantiles, etc.), we are going to need samples from this distribution. This is because with enough samples of a probability distribution, we can achieve very good approximations of summary statistics. Stan will take care of returning samples from the posterior distribution, if the log unnormalized posterior distribution is differentiable and can be expressed as follows:²

$$\log(q(\Theta|y)) = \sum_n \log(p(y_n|\Theta)) + \sum_p \log(p(\Theta_p)) \quad (6)$$

where n indicates each data point and p each parameter. In our case, this corresponds to the following:

² Incidentally, this $\log(q(\cdot))$ is the variable target in a Stan model and `lp_` in its output; see Box @ref(thm:target).

$$\begin{aligned} \log(q(\mu, \sigma | y)) = & \sum_n^{100} \log(\text{Normal}(y_n | \mu, \sigma)) + \log(\text{Normal}(\mu | 0, 20)) \\ & + \log(\text{LogNormal}(\sigma | 3, 1)) \end{aligned} \quad (7)$$

In the following sections, we'll see how we can implement this model and many others in Stan.

3 Stan syntax

A Stan program is usually saved as a `.stan` file and accessed through R (or other interfaces) and it is organized into a sequence of optional and obligatory blocks, which must be written in order. The Stan language is different from R and it is loosely based on C++; one important aspect to pay attention to is that every statement ends in a semi-colon, `;`. Blocks `{ }` do not end in semi-colons. Some functions in Stan are written in the same way as in R (e.g., `mean`, `sum`, `max`, `min`). But some are different; when in doubt, Stan documentation can be extremely helpful. In addition, the package `rstan` provides the function `lookup()` to look up for translations of functions. For example, we saw earlier that the R function `plogis()` is needed to convert from log-odds to probability space. If we need it in a Stan program, we can look for it in the following way:

`lookup(plogis)`

##	StanFunction	Arguments	ReturnType
## 262	<code>inv_logit</code>	<code>(T);(int)</code>	<code>T;real</code>
## 307	<code>log_inv_logit</code>	<code>(T);(int)</code>	<code>T;real</code>
## 323	<code>logistic_ccdf_log</code>	<code>(real, real, T);(vector, vector, vector)</code>	<code>T;real</code>
## 324	<code>logistic_cdf</code>	<code>(real, real, T);(vector, vector, vector)</code>	<code>T;real</code>
## 325	<code>logistic_cdf_log</code>	<code>(real, real, T);(vector, vector, vector)</code>	<code>T;real</code>
## 326	<code>logistic_lccdf</code>	<code>(real, real, T);(vector, vector, vector)</code>	<code>T;real</code>
## 327	<code>logistic_lcdf</code>	<code>(real, real, T);(vector, vector, vector)</code>	<code>T;real</code>

There are three columns in the output of this call. The first one indicates Stan function names, the sec-

and one their arguments with their type, and the third one the type they return. Unlike R, Stan is strict with the type of the variables.³ In order to decide on which function to use, it is necessary to look at the Stan documentation and find the function that matches our specific needs (for `plogis`, the corresponding function would be `inv_logit()`).

Another important difference with R is that every variable needs to be declared at the beginning of a block with its type (real, integer, vector, matrix, etc.).⁴ The next two sections exemplify these details through basic Stan programs.

³ In these output, there are some types that are new to the R user (but they are also used in C++): `reals` indicates that any of `real`, `real[]`, `vector`, or `row_vector`. A return type `R` with an input type `T` indicates that the type of the output of the function is the same as type of the argument.

⁴ However, this constraint is changing in newer versions.

3.1 A first simple example with Stan: Normal likelihood

Let's fit a Stan model to estimate the simple example, where we simulate data in R from a normal distribution with a true mean of 3 and a true standard deviation of 10:

```
Y <- rnorm(n = 100, mean = 3, sd = 10)
```

As mentioned earlier, Stan code is organized in blocks. The first block indicates what constitutes data for the model:

```
data {
  int<lower = 1> N; // Total number of trials
  vector[N] y; // Score in each trial
}
```

The variable of type `int` (integer) represents the number of trials. In addition to the type, some constraints can be indicated with `lower` and `upper`. In this case, `N` can't be smaller than 1. These constraints serve as a sanity check; if they are not satisfied, we get an error and the model won't run. The data are stored in a vector of length `N`, unlike R, vectors (and matrices and arrays) need to be defined with their dimensions. Comments are indicated with `//` rather than `#`.

The next block indicates the parameters of the model:


```
parameters {
  real mu;
  real<lower = 0> sigma;
}
```

The two parameters are real numbers, and `sigma` is constrained to be positive.

Finally, we indicate the prior distributions and likelihood functions in the model block:

```
model {
  // Priors:
  target += normal_lpdf(mu | 0, 20);
  target += lognormal_lpdf(sigma | 3, 1);
  // Likelihood:
  for(i in 1:N)
    target += normal_lpdf(y[i] | mu, sigma);
}
```

The variable `target` is a reserved word in Stan; every statement with `target +=` adds terms to the unnormalized *log* posterior density. We do this because adding to the unnormalized log posterior amounts to multiplying a term in the numerator of the unnormalized posterior. As explained earlier, Stan uses the shape of the unnormalized posterior to sample from the actual posterior distribution.

4 *Aside: What does target do?*

We can exemplify how `target` works with one hypothetical iteration of the sampler.

```
normal <- system.file("stan_models",
                      "normal.stan",
                      package = "bcogsci")

Y <- rnorm(n = 100, mean = 3, sd = 10)
lst_score_data <- list(y = Y, N = length(Y))
# Fit the model with the default values of number of
```

```
# chains and iterations: chains = 4, iter = 2000
fit_score <- stan(normal, data = lst_score_data)
```

In every iteration where the sampler explores the posterior space, μ and σ acquire different values (this is where the Stan algorithm stops the movement of the particle in the Hamiltonian space). Say that in an iteration, $\mu = 3.1543132$ and $\sigma = 10.6743817$. Then the following happens in the model block:

1. At the beginning of the iteration, target is zero.
2. The transformations that the sampler *automatically* does are taken into account. In our case, although σ is constrained to be positive in our model, inside Stan's sampler it is transformed to an unconstrained space amenable to Hamiltonian Monte Carlo. That is, Stan samples from an auxiliary parameter that ranges from minus infinity to infinity, which is equivalent to $\log(\sigma)$. This auxiliary parameter is then exponentiated, when it is incorporated into our model. Because of the mismatch between the constrained parameter space that we defined and the unconstrained space that it is converted to by Stan, an adjustment to the unnormalized posterior is required and added *automatically*. The reasons for this requirement are somewhat complex and will be discussed in a later lecture. In this particular case, this adjustment (which is the log absolute value of the Jacobian determinant), is equivalent to adding $\log(\sigma) = 2.3678466$ to target.
3. After `target += normal_lpdf(mu | 0, 20);` the log of the density of $Normal(0, 20)$ is evaluated at a given sample of μ (specifically 3.1543132) and this is added to target. In R, this would be `dnorm(x = 3.1543132, mean = 0, sd = 20, log = TRUE)`, which is equal to -3.9271079. Thus, target should be $-3.9271079 + 2.3678466 = -1.5592613$.
4. After `target += lognormal_lpdf(sigma | 3, 1);` we add the log of the density of $LogNormal(3, 1)$ evaluated at 10.6743817 to the previous value of the target. In R, this would be `dlnorm(x = 10.6743817, mean = 3, sd`

`= 1, log = TRUE)`, which is equal to `-3.4865941`. Thus, target should be updated to `-1.5592613 + -3.4865941 = -5.0458554`.

5. After each iteration of the for-loop in the model block, we add to the target the log density of *Normal*(3.1543132, 10.6743817) evaluated at each of the values of `Y`. In R, this would be to add `sum(dnorm(Y, 3.1543132, 10.6743817, log = TRUE))` (which is equal to `-371.8248306`) to the current value of target `-5.0458554 + -371.8248306 = -376.870686`.

This means that for the coordinates [`mu = 3.1543132`, `sigma = 10.6743817`], the height of the unnormalized posterior would be the value `exp(target) = exp(-376.870686) = 2.1239324 × 10-164`. Incidentally, the value of target is returned as `lp__` (log probability) in an object storing a fit model with Stan.

It is possible to expose the value of target, by printing `target()` inside a Stan model. The value of target after each iteration is named `lp__` in the Stan object. This can be useful for troubleshooting a problematic model.

4.1 Incrementing the log probability function (*target*) vs. using the~ notation.

In the book, we specify priors and likelihoods by explicitly incrementing the log-probability function using the following syntax:

```
target += pdf_name_lpdf(parameter | ...)
```

However, Stan also allows for specifying priors and likelihood with the so-called sampling notation with the following code.

```
parameter ~ pdf_name(...)
```

Confusingly enough a sampling statement does not perform any actual sampling, and it is meant to be a notational convenience.

The following two lines of code lead to the same behavior in Stan with respect to parameter estimation. There is, nonetheless, an important difference between them.

```
x ~ normal(mu, sigma);
target += normal_lpdf(x | mu, sigma);
```

The important difference is that the sampling notation (the notation with the $\sim$) will *drop normalizing constants*. Consider the following formula that corresponds to the log-transformed PDF of a normal distribution:

$$-\log(\sigma) - \frac{\log(2\pi)}{2} - \frac{(x - \mu)^2}{2\sigma^2} \quad (8)$$

If one uses the sampling notation, Stan will ignore the terms that don't contain parameters, such as $-\frac{\log(2\pi)}{2}$. Depending on whether the variable x , the location μ , and the scale σ are data or parameters, Stan will ignore different terms. For example, consider the case of a linear regression. The data y (taking the role of x in the previous equation) is assumed to be normally distributed with a location (μ) and scale (σ) to be estimated. In this case, only $-\frac{\log(2\pi)}{2}$ can be dropped, because both $-\log(\sigma)$ and $-\frac{(y-\mu)^2}{2\sigma^2}$ contain parameters. Another example where different terms would be dropped is the case of assigning a normal prior distribution to a parameter θ . Here, the location and scale (μ and σ) are data and θ takes the role of x in the previous equation and acts as a parameter. This means that $-\log(\sigma)$ is a constant term that can be ignored, but not $-\frac{(\theta-\mu)^2}{2\sigma^2}$ because it contains the parameter θ . Dropping constant terms does not affect parameter estimation because it only affects the unnormalized likelihood in the same way in all the parameter space. To make this more concrete, the whole plot in Figure 1 will move up or down by some constant amount, and this won't affect the Hamiltonian dynamics that determine how we sample from the posterior.

The advantage of the sampling notation is that it can be faster (when many terms are ignored), but the disadvantage is that (i) it is not compatible with the calculation

of Bayes factor with bridge sampling, or the calculation of the log-likelihood for cross validation, (ii) it misleads us into thinking that Stan is actually sampling the left term in the sampling statement, e.g., drawing y from a normal distribution in the previous example, when in fact at each step the log-probability (target) is incremented based on the parameter values determined by Hamiltonian dynamics (as explained before), and (iii) it makes it less straightforward to transition to more complex models where the sampling notation cannot be used (as in, for example, mixture models).

If one is not going to use Bayes factor with bridge sampling or cross-validation, the same speed advantage of the sampling notation can also be achieved by incrementing the log-probability with log-unnormalized probability density or mass functions (functions ending with `_lupdf` or `_lupmf`). The previous example would be translated into the following:

```
target += normal_lupdf(y | mu, sigma);
```

We didn't use curly brackets with the for-loop; this is a common practice if the for-loop has only one line, but brackets can be added and are obligatory if the for-loop spans several lines.

It's also possible to avoid the for-loop since many functions are vectorized in Stan:

```
model {
  // Priors:
  target += normal_lpdf(mu | 0, 20);
  target += lognormal_lpdf(sigma | 3, 1);
  // Likelihood:
  target += normal_lpdf(y | mu, sigma);
}
```

The for-loop and vectorized versions give us the same output: The for-loop version evaluated the log-likelihood at each value of y and added it to target. The vectorized version does not create a vector of log-likelihoods;

instead, it sums up the log-likelihood evaluated at each element of `y` and then it adds that to `target`.

The complete model looks like this:

```
data {
  int<lower = 1> N; // Total number of trials
  vector[N] y; // Score in each trial
}
parameters {
  real mu;
  real<lower = 0> sigma;
}
model {
  // Priors:
  target += normal_lpdf(mu | 0, 20);
  target += lognormal_lpdf(sigma | 3, 1);
  // Likelihood:
  target += normal_lpdf(y | mu, sigma);
}
```

You can save the above code as `normal.stan`. Alternatively, you can use the version stored in the package `bcogsci`. (Typing `?stan-normal` in the R console provides some documentation for the model.) You can access the code of the models of this book by using `system.file("stan_models", "name_of_the_model.stan", package = "bcogsci")`.

```
normal <- system.file("stan_models",
                      "normal.stan",
                      package = "bcogsci")
```

This command just points to a text file that the package `bcogsci` stores on your computer. You can open it to read the code (with any text editor, or `readLines()` in R). You'll need to compile this code and run it with `stan()`.

Stan requires the data to be in a list object in R. Below, we fit the model with the default number of chains and iterations.

```
Y <- rnorm(n = 100, mean = 3, sd = 10)
lst_score_data <- list(y = Y, N = length(Y))
```

```
# Fit the model with the default values of number of
# chains and iterations: chains = 4, iter = 2000
fit_score <- stan(normal, data = lst_score_data)
# alternatively:
# stan("normal.stan", data = lst_score_data)

traceplot(fit_score, pars = c("mu", "sigma"))

print(fit_score, pars = c("mu", "sigma"))
```

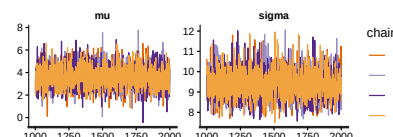


Figure 2: Trace plots.

```
## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##      mean se_mean   sd 2.5%  25%  50%  75% 97.5% n_eff Rhat
## mu    3.70    0.02 0.94 1.85 3.09 3.69 4.33 5.58 3743 1
## sigma 9.37    0.01 0.69 8.14 8.89 9.32 9.82 10.86 2666 1
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:27:49 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

```
df_fit_score <- as.data.frame(fit_score)
mcmc_hist(df_fit_score, pars = c("mu", "sigma"))
```

There are also several ways to get the samples for other summaries or customized plots, depending on whether we want a list, a data frame, or an array.

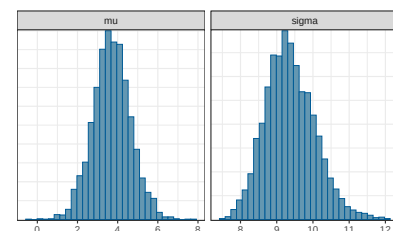


Figure 3: Histograms.

```
# extract from rstan is sometimes overwritten by
# a tidyverse version, we make sure that it's right one:
rstan::extract(fit_score) %>%
  str()

## List of 3
## $ mu : num [1:4000(1d)] 4.43 3.48 3.39 4.36 3.43 ...
## .. attr(*, "dimnames")=List of 1
## .. ..$ iterations: NULL
## $ sigma: num [1:4000(1d)] 8.4 8.51 9.97 9.91 8.75 ...
## .. attr(*, "dimnames")=List of 1
```

```
## .. ..$ iterations: NULL
## $ lp__ : num [1:4000(1d)] -370 -370 -370 -370 -369 ...
## ..- attr(*, "dimnames")=List of 1
## .. ..$ iterations: NULL

as.data.frame(fit_score) %>%
  str(list.len = 5)

## 'data.frame': 4000 obs. of 3 variables:
## $ mu : num 2.86 2.81 3.65 3.49 4.29 ...
## $ sigma: num 10.64 9.95 9.99 9.6 8.85 ...
## $ lp__ : num -371 -370 -370 -369 -369 ...

as.array(fit_score) %>%
  str()

## num [1:1000, 1:4, 1:3] 2.86 2.81 3.65 3.49 4.29 ...
## - attr(*, "dimnames")=List of 3
## ..$ iterations: NULL
## ..$ chains : chr [1:4] "chain:1" "chain:2" "chain:3" "chain:4"
## ..$ parameters: chr [1:3] "mu" "sigma" "lp__"
```

5 Another simple example: Cloze probability with Stan with the binomial likelihood

Let's fit a Stan model (`binomial_cloze.stan`) to estimate the cloze probability of a word given its context: that is, what is the probability of an upcoming word given its previous context. We want to estimate the cloze probability of “*umbrella*”, θ , given the following data: “*umbrella*” was answered 80 out of 100 trials. We assume a binomial distribution as the likelihood function, and $Beta(a = 4, b = 4)$ as a prior distribution for the cloze probability.

```
data {
  // Total number of answers
  int<lower = 1> N;
  // Number of times umbrella was answered:
  int<lower = 0, upper = N> k;
}
```



```

parameters {
  // theta is a probability, must be constrained between 0 and 1
  real<lower = 0, upper = 1> theta;
}
model {
  // Prior on theta:
  target += beta_lpdf(theta | 4, 4);
  // Likelihood:
  target += binomial_lpmf(k | N, theta);
}

```

There is only one parameter in this model, cloze probability represented with the parameter θ , which is a real number constrained between 0 and 1. Another difference between this and the previous example is that the likelihood function ends with `_lpmf` rather than with `_lpdf`. This is because Stan differentiates between distributions of continuous variables, i.e, probability density functions (PDFs), and distributions of discrete variables, i.e., probability mass functions (PMFs).

```

lst_cloze_data <- list(k = 80, N = 100)
binomial_cloze <- system.file("stan_models",
                              "binomial_cloze.stan",
                              package = "bcogsci")
fit_cloze <- stan(binomial_cloze, data = lst_cloze_data)

```

Print the summary of the posterior distribution of θ below, and show its posterior distribution graphically.

```

print(fit_cloze, pars = c("theta"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##      mean se_mean   sd 2.5%  25%  50%  75% 97.5% n_eff Rhat
## theta 0.78      0 0.04  0.7 0.75 0.78 0.81  0.85  1556    1
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:27:51 2026.
## For each parameter, n_eff is a crude measure of effective sample size,

```

```
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

```
df_fit_cloze <- as.data.frame(fit_cloze)
mcmc_dens(df_fit_cloze, pars = "theta") +
  geom_vline(xintercept = mean(df_fit_cloze$theta))
```

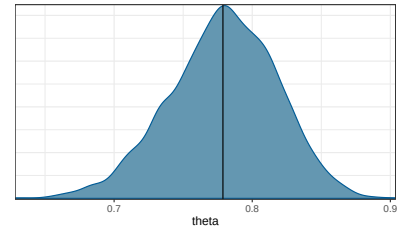


Figure 4: Posterior distribution of the theta parameter.

6 Regression models in Stan

In the following sections, we will revisit and expand on some of the examples that were fit with brms in earlier lectures.

7 A first linear regression in Stan: Does attentional load affect pupil size?

As in earlier lectures, we focus on the effect of cognitive load on one subject's pupil size with a subset of the original data. We use the following likelihood and priors.

$$\begin{aligned}
 p_size_n &\sim \text{Normal}(\alpha + c_load_n \cdot \beta, \sigma) \\
 \alpha &\sim \text{Normal}(1000, 500) \\
 \beta &\sim \text{Normal}(0, 100) \\
 \sigma &\sim \text{Normal}_+(0, 1000)
 \end{aligned} \tag{9}$$

The Stan model `pupil.stan` follows:

Because we are fitting a regression, we use the location (μ) of the likelihood function to regress `p_size` with the following equation `alpha + c_load * beta`, where both `p_size` and `c_load` are vectors defined in the data block. The following line accumulates the log-likelihood of every observation:

```
target += normal_lpdf(p_size | alpha + c_load *
beta, sigma);
```

This is equivalent to and slightly faster than the following lines:

```
for(n in 1:N)
  target += normal_lpdf(p_size[n] | alpha + c_load[n] * beta, sigma);
```

A statement that requires some explanation is the following:

```
target += normal_lpdf(sigma | 0, 1000)
        - normal_lccdf(0 | 0, 1000);
```

As in our original example in an earlier lecture, we are assuming a truncated normal distribution as a prior for σ . Not only are we setting a lower boundary to the parameter with `lower = 0`, but we are also “correcting” its prior distribution by subtracting `normal_lccdf(0 | 0, 1000)`, where `lccdf` stands for log complement of a cumulative distribution function. Once we add a lower boundary, the probability mass under half of the “regular” normal distribution should be one, that is, when we integrate from zero (rather than from minus infinity) to infinity. We need to normalize the PDF by dividing it by the difference of its CDF evaluated in the new boundaries ($a = 0$ and $b = \infty$ in our case):

$$f_{[a,b]}(x) = \frac{f(x)}{F(b) - F(a)} \quad (10)$$

where f is a PDF and F a CDF.

This equation in log-space is:

$$\log(f_{[a,b]}(x)) = \log(f(x)) - \log(F(b) - F(a)) \quad (11)$$

In Stan `log(f(x))` corresponds to `normal_lpdf(x | ...)`, and `log(F(x))` to `normal_lcdf(x | ...)`. Because in our example $b = \infty$, $F(b) = 1$, we are dealing with the complement of the log CDF evaluated at $a = 0$, $\log(1 - F(0))$, that is why we use `normal_lccdf(0 | ...)` (notice the double c; this symbol represents the complement of the CDF).

To be able to fit the model, Stan requires the data to be input as a list: First, load the data and center the dependent variable in a data frame, then create a list, and finally fit the model.

```
df_pupil <- df_pupil %>%
  mutate(c_load = load - mean(load))
```

```
ls_pupil <- list(
  p_size = df_pupil$p_size,
  c_load = df_pupil$c_load,
  N = nrow(df_pupil))
pupil <- system.file("stan_models",
                     "pupil.stan",
                     package = "bcogsci")
fit_pupil <- stan(pupil, data = ls_pupil)
```

Check the trace plots.

```
traceplot(fit_pupil, pars = c("alpha", "beta", "sigma"))
```

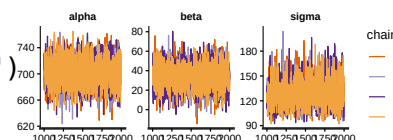


Figure 5: Traceplots.

Examine some summaries of the marginal posterior distributions of the parameters of interest:

```
print(fit_pupil, pars = c("alpha", "beta", "sigma"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##          mean se_mean    sd   2.5%   25%   50%   75%  97.5% n_eff Rhat
## alpha  701.60    0.32 20.17  662.21  688.08  701.39  714.84  741.82  4011    1
## beta   34.07    0.21 12.11   10.30  26.18  34.19  42.16  57.77  3370    1
## sigma  128.90    0.28 15.49  102.24  118.22  127.45  138.06  164.57  2971    1
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:27:52 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).
```

Plot the posterior distributions.

```
df_fit_pupil <- as.data.frame(fit_pupil)
mcmc_hist(fit_pupil, pars = c("alpha", "beta", "sigma"))
```

To determine the probability that the posterior for beta is larger than zero given the model and data, examine the proportion of samples above zero:

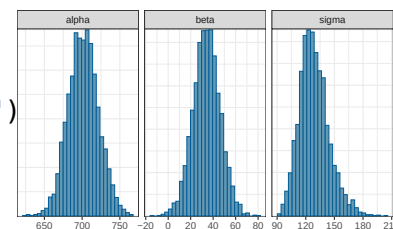


Figure 6: Histograms.

We are using df_fit_pupil and not the "raw" Stanfit object.

```
mean(df_fit_pupil$beta > 0)
```

```
## [1] 0.99675
```

To generate prior or posterior predictive distributions, we can create our own functions in R with the `purrr` function `map_dfr` (or a for-loop) as we did in an earlier chapter of the book with the function `lognormal_model_pred()`. Alternatively, we can use the generated quantities block in our model:

```
data {
  int<lower = 1> N;
  vector[N] c_load;
  int<lower= 0, upper = 1> onlyprior;
  vector[N] p_size;
}
parameters {
  real alpha;
  real beta;
  real<lower = 0> sigma;
}
model {
  // priors including all constants
  target += normal_lpdf(alpha | 1000, 500);
  target += normal_lpdf(beta | 0, 100);
  target += normal_lpdf(sigma | 0, 1000)
    - normal_lccdf(0 | 0, 1000);
  if (!onlyprior)
    target += normal_lpdf(p_size | alpha + c_load * beta, sigma);
}
generated quantities {
  array[N] real p_size_pred;
  p_size_pred = normal_rng(alpha + c_load * beta, sigma);
}
```

For most of the probability functions, there is a matching pseudorandom number generator (PRNG) with the suffix `_rng`. Here we are using the vectorized function `normal_rng`. Once `p_size_pred` is declared as an array of size `N`, the following statement generates `N` predictions (for each iteration of the sampler):

```
p_size_pred = normal_rng(alpha + c_load * beta, sigma);
```

At the moment not all the PRNG are vectorized, but the ones that are only allow for arrays and, confusingly enough, not vectors. We define an array by indicating array, between brackets, the length of each dimension, then the type, and finally the name of the variable. For example to define an array of real numbers with three dimension of length 6, 7, and 10 we write `array[6, 7, 10] real var`.⁵ Vectors and matrices are also valid types for an array.

⁵ The notation for arrays has changed in recent versions, the previous notation would have been `real[6, 7, 10] var`.

We also included a data variable called `onlyprior`, this is an integer that can only be set to 1 (TRUE) or 0 (FALSE). When `onlyprior = 1`, the likelihood is omitted from the model, `p_size` is ignored, and `p_size_pred` is the prior predictive distribution. When `onlyprior = 0`, the likelihood is incorporated in the model (as it is in the original code `pupil.stan`) using `p_size`, and `p_size_pred` is the posterior predictive distribution.

If we want posterior predictive distributions, we fit the model to the data and set `onlyprior = 0`, if we want prior predictive distributions, we sample from the priors and set `onlyprior = 1`. Then we use `bayesplot` functions to visualize predictive checks.

For posterior predictive checks, we would do the following:

```
ls_pupil <- list(onlyprior = 0,
                p_size = df_pupil$p_size,
                c_load = df_pupil$c_load,
                N = nrow(df_pupil))
pupil_gen <- system.file("stan_models",
                        "pupil_gen.stan",
                        package = "bcogsci")
fit_pupil <- stan(file = pupil_gen, data = ls_pupil)
```

Store the predicted pupil sizes in `yrep_pupil`. This variable contains an $N_{samples} \times N_{observations}$ matrix, that is, each row of the matrix is a draw from the posterior predictive distribution, i.e., a vector with one element for each of the

data points in `y`.

```
yrep_pupil <- extract(fit_pupil)$p_size_pred
dim(yrep_pupil)
## [1] 4000 41
```

Predictive checks functions in `bayesplot` (starting with `ppc_`) require a vector with the observations in the first argument and a matrix with the predictive distribution as its second argument. As an example, we use an overlay of densities and we draw only 50 elements (that is 50 predicted data sets).

```
ppc_dens_overlay(df_pupil$p_size, yrep = yrep_pupil[1:50])
```

For prior predictive distributions, we simply set `onlyprior = 1`. The observations (`p_size`) are ignored by the model, but are required by the data block in Stan. If we haven't collected data yet, we could include a vector of zeros.

```
ls_pupil_prior <- list(onlyprior = 1,
  p_size = df_pupil$p_size,
  # or: p_size = rep(0, nrow(df_pupil)),
  c_load = df_pupil$c_load,
  N = nrow(df_pupil))
prior_pupil <- stan(pupil_gen, data = ls_pupil_prior,
  control = list(adapt_delta = 0.99))
```

To avoid divergent transitions, increase the `adapt_delta` parameter's default value from 0.8 to 0.99. It is important to highlight that we cannot safely ignore the warnings of the above model, even if we are not fitting data. This is so because in practice one is still sampling a density using Hamiltonian Monte Carlo, and thus the prior sampling process can break in the same ways as the posterior sampling process. Prior predictive distributions as the one shown here can be plot with `ppd_dens_overlay()` (and in general with functions starting with `ppd_` which don't require an argument with data).

```
yrep_prior_pupil <- extract(prior_pupil)$p_size_pred
ppd_dens_overlay(yrep_prior_pupil[1:50, ])
```

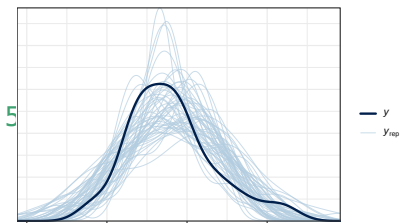


Figure 7: Density overlay plot.

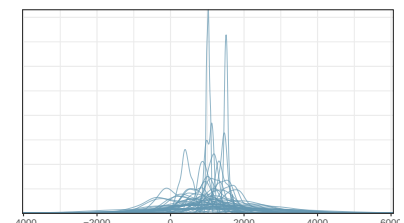


Figure 8: Density overlay plot.

7.1 *Matrix, vector, or array in Stan?*

Stan contains three basic linear algebra types, `vector`, `row_vector`, and `matrix`. But Stan also allows for building arrays of any dimension from any type of element (integer, real, etc.). This means that there are several ways to define one-dimensional N -sized containers of real numbers,

```
array[N] real a;
vector[N] a;
row_vector[N] a;
```

as well as, two-dimensional $N_1 \times N_2$ -sized containers of real numbers:

```
array[N1, N2] real m;
matrix[N1, N2] m;
array[N1] vector[N2] b;
array[N1] row_vector[N2] b;
```

These distinctions affect either what we can do with these variables, or the speed of our model, and sometimes are interchangeable. Matrix algebra is only defined for (row) vectors and matrices, that is we cannot multiply arrays. The following line requires all the one-dimensional containers (`p_size` and `c_load`) to be defined as vectors (or row_vectors):

```
vector[N] mu = alpha + c_load * beta;
```

Many “vectorized” operations are also valid for arrays, that is, `normal_lpdf`, accepts (row) vectors (as we did in our code) or arrays as in the next example. There is of course no point in converting a vector to an array as follows, but this shows that Stan allows both type of one-dimensional containers.

```
array[N] real mu = to_array_1d(alpha + c_load * beta);
target += normal_lpdf(p_size | mu, sigma);
```

By contrast, the outcome of “vectorized” pseudorandom number generator (`_rng`) functions can only be

stored in an array. The following example shows the only way to vectorize this type of function:

```
array[N] real p_size_pred = normal_rng(alpha + c_load * beta,
                                         sigma);
```

Alternatively, one can always use a for-loop, and it won't matter if `p_size_pred` is an array or a vector:

```
vector[N] p_size_pred;
for(n in 1:N)
  p_size_pred[n] = normal_rng(alpha + c_load[n] * beta, sigma);
```

See also Stan's user's guide section on matrices, vector, and arrays, chapter 18 of the User's guide.

7.2 Interactions in Stan: Does attentional load interact with trial number affecting pupil size?

We'll expand the previous model to also include the effect of (centered) trial and its interaction with cognitive load on one subject's pupil size. Our new likelihood will look as follows:

$$p_size_n \sim \text{Normal}(\alpha + c_load_n \cdot \beta_1 + c_trial \cdot \beta_2 + c_load \cdot c_trial \cdot \beta_3, \sigma) \quad (12)$$

Define priors for all the new β s. Since we don't have more information about the new predictors, they are sampled from identical prior distributions:

$$\begin{aligned} \alpha &\sim \text{Normal}(1000, 500) \\ \beta_1 &\sim \text{Normal}(0, 100) \\ \beta_2 &\sim \text{Normal}(0, 100) \\ \beta_3 &\sim \text{Normal}(0, 100) \\ \sigma &\sim \text{Normal}_+(0, 1000) \end{aligned} \quad (13)$$

The following Stan model, `pupil_int1.stan`, is the direct translation of the new priors and likelihood.

```
data {
  int<lower = 1> N;
```

```

vector[N] c_load;
vector[N] c_trial;
vector[N] p_size;
}
parameters {
  real alpha;
  real beta1;
  real beta2;
  real beta3;
  real<lower = 0> sigma;
}
model {
  // priors including all constants
  target += normal_lpdf(alpha | 1000, 500);
  target += normal_lpdf(beta1 | 0, 100);
  target += normal_lpdf(beta2 | 0, 100);
  target += normal_lpdf(beta3 | 0, 100);
  target += normal_lpdf(sigma | 0, 1000)
    - normal_lccdf(0 | 0, 1000);
  target += normal_lpdf(p_size | alpha + c_load * beta1 +
    c_trial * beta2 +
    c_load .* c_trial * beta3, sigma);
}

```

When there are matrices or vectors involved, $*$ indicates matrix multiplication whereas $.*$ indicates element-wise multiplication; in R $%%$ indicates matrix multiplication whereas $*$ indicates element-wise multiplication.

There is, however, an alternative notation that can simplify our code. In the following likelihood, p_size is a vector of N observations (in this case 41), X is the model matrix with a dimension of $N \times N_{pred}$ (in this case 41×3), and β a vector of N_{pred} (in this case, 3) rows. Assuming that β is a vector, we indicate with one line that each parameter β_n is sampled from identical prior distributions.

$$\begin{aligned}
 p_size &\sim \text{Normal}(\alpha + X \cdot \beta, \sigma) \\
 \beta &\sim \text{Normal}(0, 100) \\
 \sigma &\sim \text{Normal}_+(0, 1000)
 \end{aligned}
 \tag{14}$$

The translation into Stan code is the following:

```
data {
  int<lower = 1> N;
  int<lower = 0> K;    // number of predictors
  matrix[N, K] X;    // model matrix
  vector[N] p_size;
}
parameters {
  real alpha;
  vector[K] beta;
  real<lower = 0> sigma;
}
model {
  // priors including all constants
  target += normal_lpdf(alpha | 1000, 500);
  target += normal_lpdf(beta | 0, 100);
  target += normal_lpdf(sigma | 0, 1000)
    - normal_lccdf(0 | 0, 1000);
  target += normal_lpdf(p_size | alpha + X * beta, sigma);
}
```

For some likelihood functions, Stan provides a more efficient implementation of the linear regression than the one manually written in the previous code. It's critical to understand that, in general, a more efficient implementation should not only be faster, but should also achieve the same number of effective samples (or more) than a less efficient implementation (and should also show convergence). In this case, we can achieve that using `_glm` functions. We can replace the last line with the following statement (the order of the arguments is important):⁶

```
target += normal_id_glm_lpdf(p_size | X, alpha, beta, sigma);
```

The most optimized model, `pupil_int.stan`, includes this last statement. We prepare the data as follows: First create a centered version of `trial`, `c_trial` and load `c_load`, then use the function `model.matrix` to create the `X` matrix that contains in each column our predictors and omits the intercept with `0 +`.

⁶ An extra boost in efficiency can be obtained in regular regressions where `X` and `Y` are data (rather than parameters as in cases of missing data or measurement error), since this function can be executed on a GPU.

```

df_pupil <- df_pupil %>%
  mutate(c_trial = trial - mean(trial),
         c_load = load - mean(load))
X <- model.matrix(~ 0 + c_load * c_trial, df_pupil)
ls_pupil_X <- list(p_size = df_pupil$p_size,
                  X = X,
                  K = ncol(X),
                  N = nrow(df_pupil))

pupil_int <- system.file("stan_models",
                        "pupil_int.stan",
                        package = "bcogsci")
fit_pupil_int <- stan(pupil_int, data = ls_pupil_X)
print(fit_pupil_int, pars = c("alpha", "beta", "sigma"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##          mean se_mean    sd  2.5%   25%   50%   75%  97.5% n_eff Rhat
## alpha    699.66    0.23 16.43 667.55 689.03 699.64 710.44 732.13  5013    1
## beta[1]   31.41    0.14  9.81  11.91  24.93  31.53  37.93  50.79  5065    1
## beta[2]  -5.82    0.02  1.39  -8.51  -6.75  -5.81  -4.91  -3.13  4801    1
## beta[3]  -1.83    0.01  0.84  -3.51  -2.38  -1.83  -1.26  -0.20  4800    1
## sigma   104.72    0.20 12.74  83.58  95.71 103.47 112.42 132.88  4015    1
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:27:56 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).

```

Here, we plot here the 95% CrI of the parameters of interest. We use `regex_pars`, rather than `pars`, because we want to capture `beta[1]`, `beta[2]`, and `beta[3]`; `regex_pars` use regular expressions to select the parameters (for information about regular expressions in R see <https://stat.ethz.ch/R-manual/R-devel/library/base/html/regex.html>)

```

df_fit_pupil_int <- as.data.frame(fit_pupil_int)
mcmc_intervals(fit_pupil_int,

```

```

regex_pars = "beta",
prob_outer = .95,
prob = .8,
point_est = "mean")

```

8 Logistic regression in Stan: Does set size and trial affect free recall?

We revisit and expand on the analysis presented in earlier lectures. In this example, we will investigate whether the length of a list and trial number affect the probability of correctly recalling a word.

As before, we assume a Bernoulli likelihood with a logit link function, and the following priors (recall that the logistic function is the inverse of the logit).

$$\begin{aligned}
 correct_n &\sim \text{Bernoulli}(\text{logistic}(\alpha + X \cdot \beta)) \\
 \alpha &\sim \text{Normal}(0, 1.5) \\
 \beta &\sim \text{Normal}(0, 0.1)
 \end{aligned}
 \tag{15}$$

Where β is a vector of size $K = 2$, $[\beta_0, \beta_1]$. Below in `recall.stan` we present the most efficient way to code this in Stan.

```

data {
  int<lower = 1> N;
  int<lower=0> K;    // number of predictors
  matrix[N, K] X;   // model matrix
  array[N] int correct;
}
parameters {
  real alpha;
  vector[K] beta;
}
model {
  // priors including all constants
  target += normal_lpdf(alpha | 0, 1.5);
  target += normal_lpdf(beta | 0, .1);
  target += bernoulli_logit_glm_lpmf(correct | X, alpha, beta);
}

```

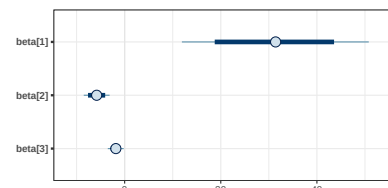


Figure 9: Credible intervals for the intercept and slope parameters in the model.

The dependent variable, `correct`, is an array of integers rather than a vector; this is because vectors are always composed of real numbers, but the Bernoulli likelihood only accepts the integers 1 or 0. As in the previous example, we are taking advantage of the `_glm` functions. A less efficient but more transparent option would be to replace the last statement with:

```
target += bernoulli_logit_lpmf(correct | alpha + X * beta);
```

We might want to use `bernoulli_logit_lpmf` if we want to define a non-linear relationship between the predictors that are outside the generalized linear model framework. One example would be the following:

```
target += bernoulli_logit_lpmf(correct | alpha + exp(X * beta));
```

Another more flexible possibility when we want to indicate a Bernoulli likelihood is to use `bernoulli_lpmf` and add the link manually. The last statement of `recall.stan` would become the following:

```
target += bernoulli_lpmf(correct | inv_logit(alpha + X * beta));
```

The function `bernoulli_lpmf` can be useful if one wants to try other link functions; see exercise [@ref\(exr:linkfunction\)](#).

Finally, the most transparent form (but less efficient) would be the following for-loop:

```
for (n in 1:N)
  target += bernoulli_lpmf(correct[n] | inv_logit(alpha + X[n] * beta));
```

To fit the model as `recall.stan`, prepare the data by centering the trial number first:

```
df_recall <- df_recall %>%
  mutate(c_set_size = set_size - mean(set_size),
         c_trial = trial - mean(trial))
```

Next, we create the model matrix, X , and prepare the data as a list. As before, we exclude the intercept from the matrix X using `0 + ...`. This is because the Stan code that we are using already takes into account that the first column in the model matrix is going to be a vector of ones.

```

X <- model.matrix(~ 0 + c_set_size * c_trial, df_recall)
ls_recall <- list(correct = df_recall$correct,
                  X = X,
                  K = ncol(X),
                  N = nrow(df_recall))

recall <- system.file("stan_models",
                      "recall.stan",
                      package = "bcogsci")
fit_recall <- stan(recall, data = ls_recall)

```

After fitting the model we can print and plot summaries of the posterior distribution.

```

print(fit_recall, pars = c("alpha", "beta"))

## Inference for Stan model: anon_model.
## 4 chains, each with iter=2000; warmup=1000; thin=1;
## post-warmup draws per chain=1000, total post-warmup draws=4000.
##
##          mean se_mean   sd  2.5%  25%   50%   75%  97.5% n_eff Rhat
## alpha    1.99    0.01 0.33   1.38  1.77  1.98  2.20  2.67  3312   1
## beta[1] -0.19    0.00 0.08  -0.35 -0.24 -0.19 -0.13 -0.03  4064   1
## beta[2] -0.02    0.00 0.04  -0.09 -0.05 -0.02  0.00  0.05  3615   1
## beta[3]  0.00    0.00 0.02  -0.03 -0.01  0.00  0.01  0.04  3245   1
##
## Samples were drawn using NUTS(diag_e) at Thu Mar 26 09:27:57 2026.
## For each parameter, n_eff is a crude measure of effective sample size,
## and Rhat is the potential scale reduction factor on split chains (at
## convergence, Rhat=1).

df_fit_recall <- as.data.frame(fit_recall)
mcmc_intervals(df_fit_recall,
               regex_pars = "beta",
               prob_outer = .95,
               prob = .8,
               point_est = "mean")

```

We might want to communicate the posterior in proportions rather than in log-odds (as seen in the parameters beta). We can do this in R manipulating the data

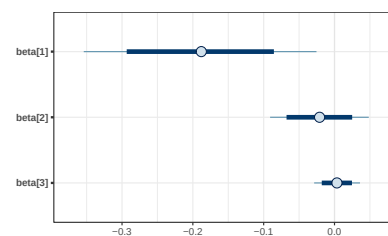


Figure 10: Credible intervals.

frame `df_fit_recall`, or extracting the samples with `extract(fit_recall)`. Another alternative presented here is by using the generated quantities block. To make the code more compact we declare the type of each variable and store its content in the same line in `recall_prop.stan`.

```
generated quantities {
  real average_accuracy = inv_logit(alpha);
  vector[K] change_acc = inv_logit(alpha) - inv_logit(alpha - beta);
}
```

Recall that due to the non-linearity of the scale, the effects depend on the average accuracy, and the set size and trial that we start from (in this case we are examining the change of one unit from the average set size and the average trial).

```
recall_prop <- system.file("stan_models",
                           "recall_prop.stan",
                           package = "bcogsci")
fit_recall <- stan(recall_prop, data = ls_recall)
```

The plot now shows how the average accuracy deteriorates when the subject is exposed to a set size larger than the average by one, a trial further than the middle one, and the interaction of both.

```
df_fit_recall <- as.data.frame(fit_recall) %>%
  rename(set_size = `change_acc[1]`,
         trial = `change_acc[2]`,
         interaction = `change_acc[3]`)
mcmc_intervals(df_fit_recall,
               pars = c("set_size", "trial", "interaction"),
               prob_outer = .95,
               prob = .8,
               point_est = "mean") +
  xlab("Change in accuracy")
```

The figure shows that our model is estimating that by increasing the set size by one unit, the recall accuracy of the single subject deteriorates by 2%. In contrast, there is hardly any trial effect or interaction between trial and set size.

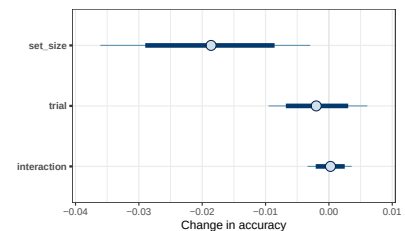


Figure 11: Effect of set size, trial, and their interaction on the average accuracy of recall.